



파머완 5장 - 회귀 (5.1장 ~ 5.8장)

01. 회귀 소개

- 회귀 분석 (Regression) : 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 회귀 - 여러 개의 독립 변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법
- 머신러닝에서의 회귀 : 주어진 feature와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것

[회귀 유형]

1. 회귀 계수의 선형 여부 → 선형 회귀(선형), 비선형 회귀(비선형)
 2. 독립 변수의 개수 → 단일 회귀 (1개), 다중 회귀 (여러 개)
- 지도 학습 : 분류 (예측값이 이산형 클래스), 회귀 (연속형 숫자 값) 로 나뉨

[선형 회귀]

: 실제 값과 예측값의 차이를 최소화하는 직선형 회귀선을 최적화하는 방식

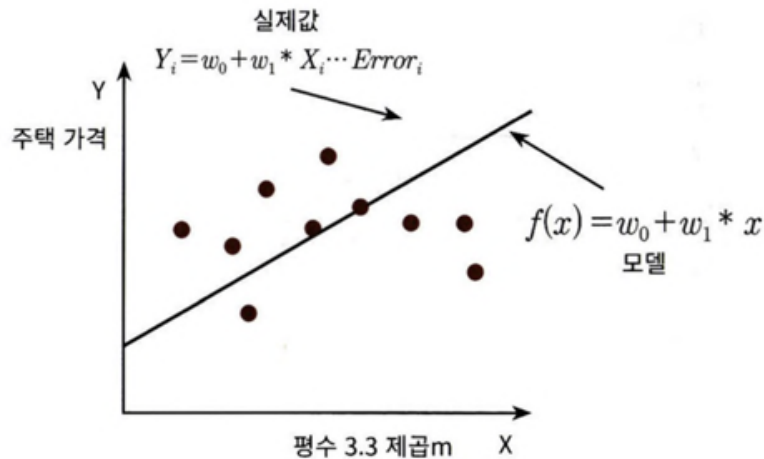
- 규제 (regularization)에 따라 유형 나뉨

일반 선형 회귀	예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화할 수 있도록 회귀 계수를 최적화하며, 규제(Regularization)를 적용하지 않은 모델
릿지(Ridge)	선형 회귀에 L2 규제를 추가한 회귀 모델로, L2 규제를 적용 (*L2 규제는 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제 모델)
라쏘(Lasso)	선형 회귀에 L1 규제를 적용한 방식 (L2 규제가 회귀 계수 값의 크기를 줄이는 데 반해, L1 규제는 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 하는 것)
엘라스틱넷 (ElasticNet)	L2, L1 규제를 함께 결합한 모델. 주로 피처가 많은 데이터 세트에서 적용되며, L1 규제에 피처의 개수를 줄임과 동시에 L2 규제에 계수 값의 크기를 조정
로지스틱 회귀 (Logistic Regression)	분류에 사용되는 선형 모델 로지스틱 회귀는 매우 강력한 분류 알고리즘이며, 일반적으로 이진 분류뿐만 아니라 희소 영역의 분류(EX-텍스트 분류)와 같은 영역에서 뛰어난 예측 성능을 보임

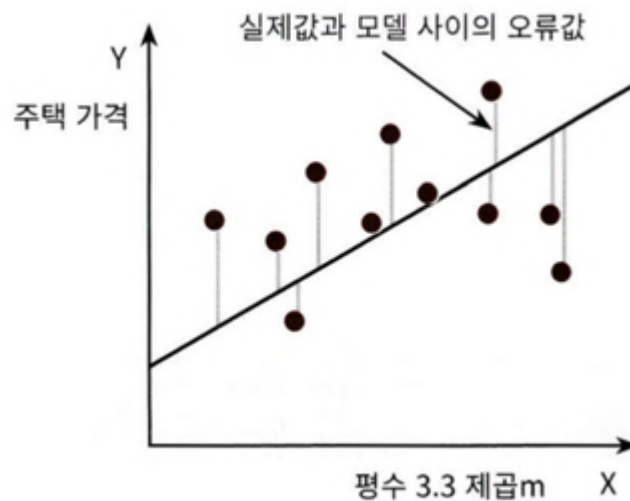
02. 단순 선형 회귀를 통한 회귀 이해

단순 선형 회귀

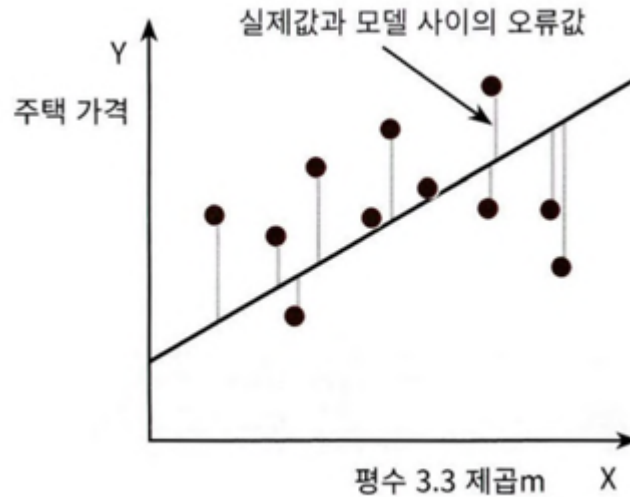
- 독립변수 1개, 종속 변수 1개
- EX) 주택 가격과 주택의 크기 간의 관계 (주택의 크기가 크면 가격이 높아지는 경향) → 주택 가격은 주택 크기에 대해 선형의 관계로 표현 가능



- 예측값($\hat{Y}$) : $w_0 + w_1 * X$
- w_1 (기울기), w_0 (절편) : 회귀 계수
- 잔차 : 실제 값과 회귀 모델의 차이에 따른 오류 값
- 최적의 회귀 모델을 만든다 → 전체 데이터의 잔차 합이 최소가 되는 모델을 만들 (= 오류 값 합이 최소가 될 수 있는 최적의 회귀 계수를 찾는다)



- 오류 값은 +,- 둘 다 가능 → 오류 합을 계산할 때, 절댓값을 취해 더하는 방식 (Mean absolute error) , 오류 값의 제곱을 구해서 더하는 방식 (RSS, Residual Sum of Square)



→ RSS는 변수가 w_0, w_1 인 식으로 표현 가능, RSS를 최소로 하는 회귀 계수를 학습을 통해서 찾는 것이 머신러닝 기반 회귀의 핵심 !

! RSS는 회귀식의 독립변수 X , 종속변수 Y 가 중심 변수가 아니라 w 변수 (회귀 계수)가 중심 변수임을 인지해야 함.

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N 까지)

- RSS (비용함수, 손실함수, loss function) → 데이터를 계속 학습하며 비용 함수가 반환하는 값을 지속해서 감소시키고, 최종적으로 더 이상 감소하지 않는 최소의 오류 값을 구함.

03. 비용 최소화하기 - 경사 하강법 (Gradient Descent) 소개

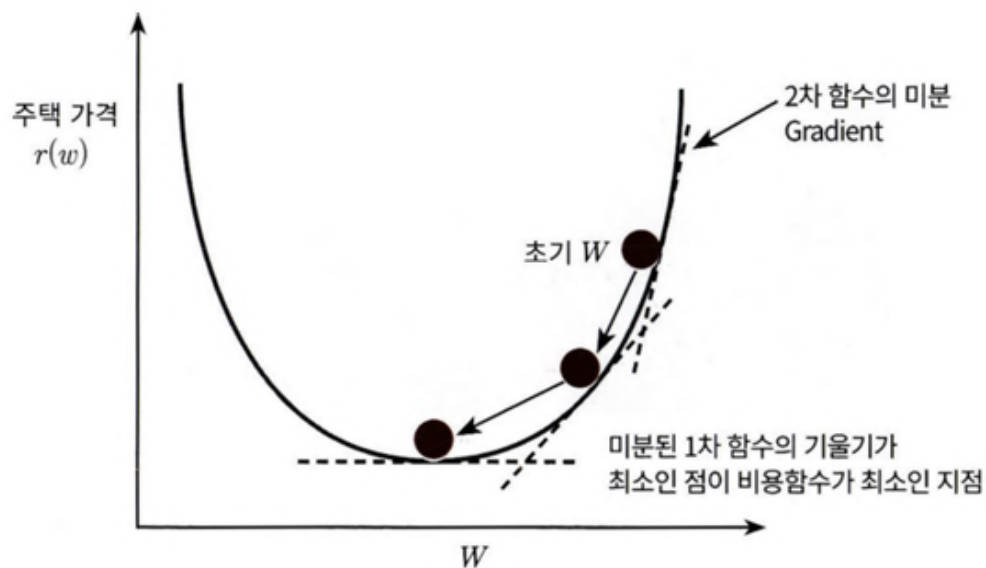
- 비용함수가 최소가 되는 W 파라미터를 구하는 방법 ?

→ 경사 하강법 : 점진적으로 반복적인 계산을 통해 w 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터를 구하는 방식

[경사 하강법]

- "어떻게 하면 오류가 작아지는 방향으로 W 값을 보정할 수 있을까?"
- 비용 함수가 포물선 형태의 2차 함수인 경우

→ 경사 하강법은 최초 w 에서부터 미분 적용, 이 미분 값이 계속 감소하는 방향으로 순차적으로 W 를 업데이트 → 더 이상 미분된 1차 함수의 기울기가 감소하지 않는 지점을 비용 함수가 최소인 지점으로 간주, 그때의 w 를 반환



*RSS(비용 함수)를 $R(w)$ 로 지칭

$$R(w) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

→ $R(w)$ 를 미분해서 미분함수의 최솟값을 구할 때, w_0 & w_1 각 변수에 편미분 적용

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

위의 w_1 , w_0 의 편미분 결과값을 반복적으로 보정하며 w_1 , w_0 값을 업데이트

→ $R(w)$ 가 최소가 되는 w_1 , w_0 의 값을 구할 수 있음

- 학습률 : 편미분 값이 너무 클 수 있기 때문에 보정계수를 곱하는 것

- Step 1: w_1, w_0 를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.

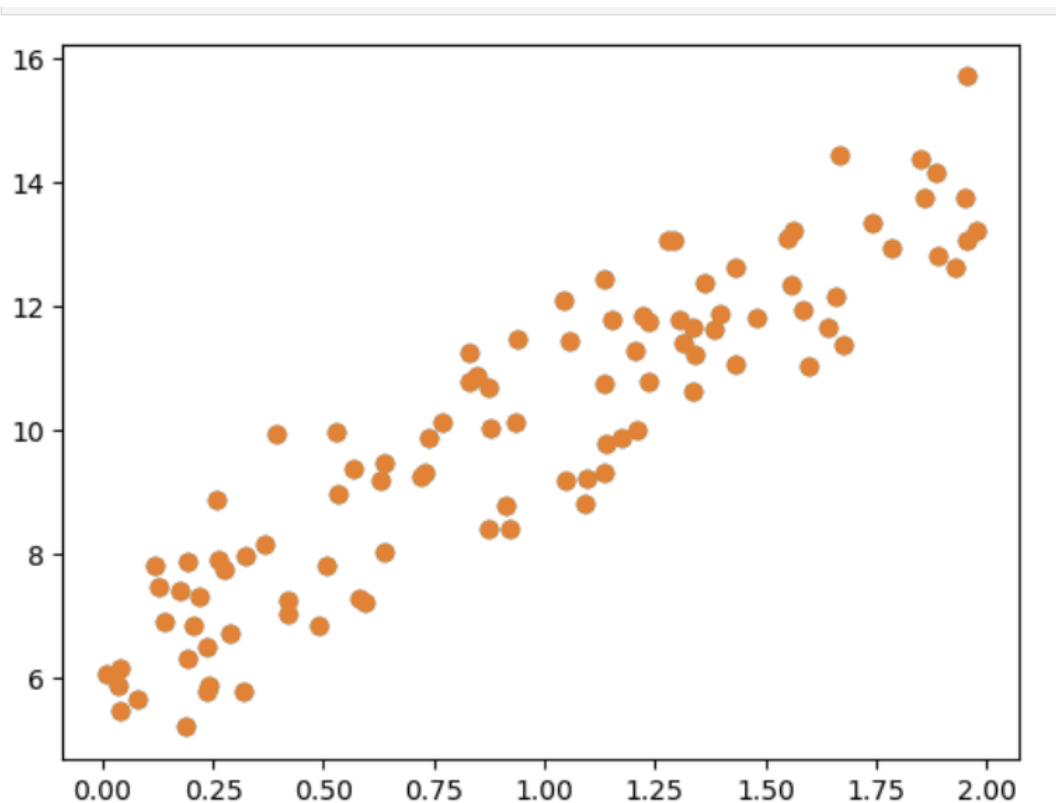
- Step 2: w_1 을 $w_1 + \eta \frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$, w_0 을 $w_0 + \eta \frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$ 으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.

- Step 3: 비용 함수가 감소하는 방향으로 주어진 횟수만큼 Step 2를 반복하면서 w_1 과 w_0 를 계속 업데이트합니다.

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(0)
# y = 4X + 6을 근사(w1=4, w0=6). 임의의 값은 노이즈를 위해 만듦.
X = 2 * np.random.rand(100,1)
y = 6 + 4*X+np.random.randn(100,1)

# X, y 데이터 세트 산점도
plt.scatter(X, y)
plt.show()
```



gradient_descent() 함수로 경사하강법 구현

gradient_descent_steps() 함수: get_weight_updates()을 경사 하강 방식으로 반복적으로 수행하여 w1과 w0를 업데이트

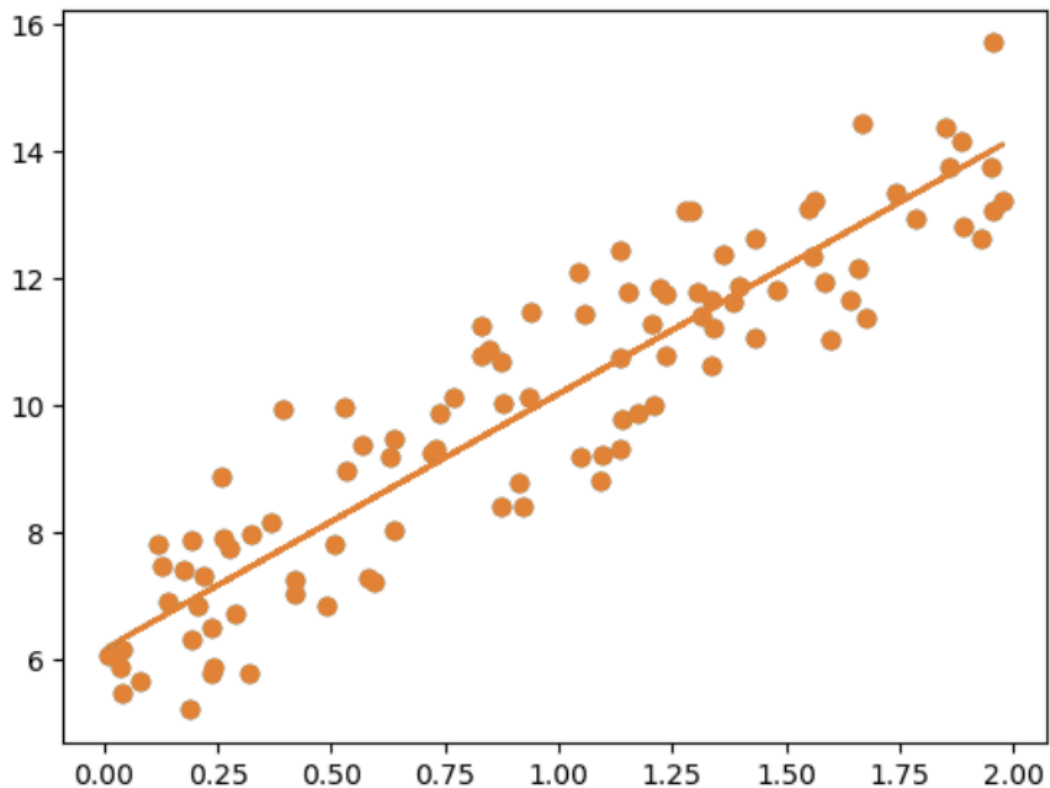
```
def get_cost(y, y_pred):
    N = len(y)
    cost = np.sum(np.square(y-y_pred))/N
    return cost

w1, w0 = gradient_descent_steps(X, y, iters = 1000)
print("w1:{0:.3f} w0:{1:.3f}".format(w1[0, 0], w0[0, 0]))
y_pred = w1[0, 0] * X + w0
print('Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))
```

```
w1:4.022 w0:6.162
Gradient Descent Total Cost:0.9935
```

→ 실제 선형식 $y = 4x + 6$ 과 유사하게 w1:4.022, w0:6.162 도출 (예측 오류 비용 : 0.9935)

```
plt.scatter(X,y)
plt.plot(X, y_pred)
plt.show()
```



→ 경사하강법을 이용해 회귀선이 잘 만들어졌음을 확인

- 확률적 경사 하강법 (Stochastic Gradient Descent) : 전체 입력 데이터로 파가 업데이트되는 값을 계산X, 일부 데이터만 이용해 파가 업데이트되는 값을 계산하므로 경사 하강법에 비해서 빠른 속도를 보장
- stochastic_gradient_descent_steps() 함수는 gradient_descent_steps()와 거의 비슷 ↔ 전체 X, y 데이터에서 랜덤하게 batch_size만큼 데이터를 추출해 이를 기반으로 w1_update, w0_update를 계산하는 부분만 다름

```
#stochastic_gradient_descent_steps()를 이용해 w1, w0 및 예측 오류 비용을 계산

w1, w0 = stochastic_gradient_descent_steps(X, y, iters=1000)
print("w1:", round(w1[0, 0], 3), "w0:", round(w0[0, 0], 3))
y_pred = w1 [0, 0] * X + w0
print('Stochastic Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))

w1: 4.028 w0: 6.156
Stochastic Gradient Descent Total Cost:0.9937
```

- (미니배치) 확률적 경사 하강법으로 구한 w_0, w_1 결과는 경사하강법으로 구한 w_1, w_0 과 큰 차이가 없으며, 예측 오류 비용 또한 큰 예측 성능상의 차이 X

-피처가 여러 개인 경우에는 어떻게 회귀 계수를 도출할 수 있을까?

→ 1개인 경우를 확장해 유사하게 도출

$$\hat{Y} = X_{mat} \begin{matrix} \text{Feature} & \text{Feature} & \dots & \text{Feature} \\ 1 & 2 & & M \end{matrix} \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix} \star \begin{bmatrix} w_1 & w_2 & \dots & w_m \end{bmatrix}^T + w_0$$

내적

w_0 를 Weight의 배열인 W 안에 포함시키기 위해서 X_{mat} 의 맨 처음 열에 모든 데이터의 값이 1인 피처 Feat 0을 추가하겠습니다. 이제 회귀 예측값은 $\hat{Y} = X_{mat} \star W^T$ 와 같이 도출할 수 있습니다.

$$\hat{Y} = X_{mat} \begin{matrix} \text{Feat} & \text{Feat} & \text{Feat} & \dots & \text{Feat} \\ 0 & 1 & 2 & & M \end{matrix} \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1m} \\ 1 & x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix} \star \begin{bmatrix} w_0 & w_1 & w_2 & \dots & w_m \end{bmatrix}^T$$

내적

$\hat{Y} = X_{mat} \star W^T$

04. 사이킷런 Linear Regression을 이용한 보스턴 주택 가격 예측

[LinearRegression 클래스 - Ordinary Least Squares]

- 예측값과 실제값의 RSS를 최소화해 OLS 추정 방식으로 구현한 클래스
- 메서드로 X, y 배열을 입력받음 → 회귀계수인 W 를 `coef_`에 저장

```
class sklearn.linear_model.LinearRegression(fit_intercept = True, normalize =
                                             copy_X = True, n_jobs = 1)
```


fit_intercept	불린 값으로, 디폴트는 True. Intercept 값 계산 여부 결정 (False → intercept 사용X, 0으로 지정)
normalize	불린 값으로, 디폴트는 False. fit_intercept가 False인 경우에는 무시됨. True이면 회귀를 수행하기 전에 입력 데이터 세트를 정규화
coef_	fit() 메서드를 수행했을 때 회귀계수가 배열 형태로 저장하는 속성
intercept_	intercept 값

- OLS 기반 회귀 계수 계산은 입력 피처의 독립성에 많은 영향을 받음

→ 다중 공선성 : 피처 간의 상관관계가 매우 높은 경우 분산이 매우 커져서 오류에 매우 민감해짐

[회귀 평가 지표]

: 실제 값과 회귀 예측값의 차이 값을 기반으로 한 지표가 중심

→ +, -가 있기 때문에 오류의 절댓값 평균이나 제곱, 또는 제곱한 뒤 다시 루트를 씌운 평균 값을 구함

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$

MAE	metrics.mean_absolute_error
MSE	metrics.mead_squared_error
RMSE	metrics.mean_squared_error 사용하되 squared 파라미터는 false
MSLE	metrics.mearn_squared_log_error
R2	metrics.r2_score

!! 실제 값과 예측값의 오류 차이를 기반으로 하는 회귀 평가 지표의 경우 값이 커지면 나쁜 모델 → 사이킷런의 Scoring 함수에 일반적으로 반영하려면 보정이 필요

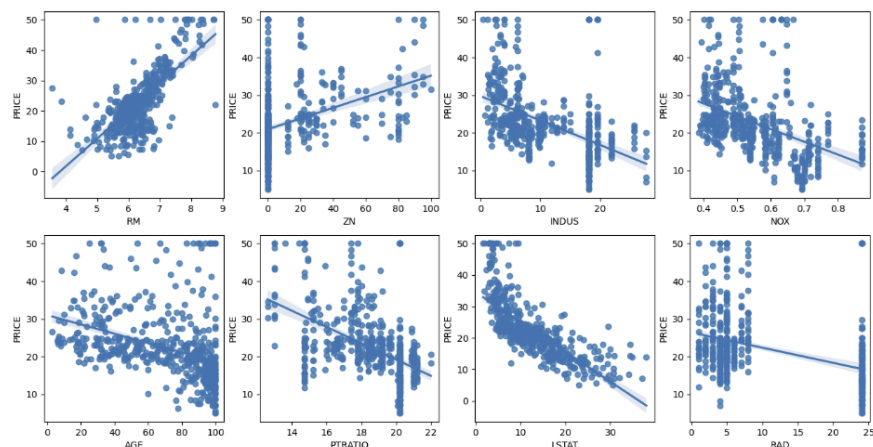
⇒ -1을 원래의 평가 지표 값에 곱해서 음수를 만들어 작은 오류 값이 더 큰 숫자로 인식하게 함.

[LinearRegression을 이용한 보스턴 주택 가격 회귀 구현]

→ load_boston 이 데이터셋의 윤리적인 문제로 삭제됨 : 원본 데이터 (url)를 불러와서 사용.

- Seaborn의 regplot () : X,Y축 산점도 & 선형회귀직선
- matplotlib.subplots() : 여러 개의 그래프를 한 번에 표현하기 위해 자주 사용됨 → 각 ax마다 칼럼과 PRICE 관계 표현함

```
fig, axs = plt.subplots(figsize = (16,8), ncols = 4, nrows = 2)
lm_features = ['RM', 'ZN', 'INDUS', 'NOX', 'AGE', 'PTRATIO', 'LSTAT', 'RAD']
for i, feature in enumerate(lm_features):
    row = int(i/4)
    col = i%4
    #시본의 regplot을 이용해 산점도와 선형 회귀 직선을 함께 표현
    sns.regplot(x = feature, y = 'PRICE', data = bostonDF, ax= axs[row][col])
```



⇒ RM, LSTAT의 PRICE 영향도가 가장 두드러짐

: RM(방 개수)은 양 방향의 선형성이 가장 크며, 이는 방의 크기가 클수록 가격이 증가하는 모습을 확연히 보여줌

:LSTAT(하위계층의 비율)은 음 방향의 선형성이 가장 크며, 이는 LSTAT이 적을 수록 PRICE가 증가하는 모습이 나타남

- coef_ , intercept_ : 회귀 계수와 절편 값
- coef_ 속성은 회귀 계수값만 가지고 있으므로 이를 피처별 회귀 계수 값으로 다시 매핑 & 높은 값 순으로 출력 (sort_values)

5개의 폴드 세트에서 cross_val_score()를 이용해 교차 검증으로 MSE와 RMSE를 측정

- `cross_val_score()`에서 반환된 값에 다시 -1을 곱해야 양의 값인 원래 모델에서 계산된 MSE 값이 됨

```
from sklearn.model_selection import cross_val_score

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)
lr = LinearRegression()

#cross_val_score()로 5 폴드 세트의 MSE를 구한 뒤, 이를 기반으로 다시 RMSE 구함
neg_mse_scores = cross_val_score(lr, X_data, y_target, scoring = "neg_mean_
rmse_scores = np.sqrt(-1*neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

#cross_val_score(scoring="negjmean_squared_error")로 반환된 값은 모두 음수
print('5 folds 의 개별 Negative MSE scores:', np.round(neg_mse_scores, 2))
print('5 folds 의 개별 RMSE scores:', np.round(rmse_scores, 2))
print('5 folds 의 평균 RMSE: {0:.3f}'.format(avg_rmse))
```

```
5 folds 의 개별 Negative MSE scores: [-12.46 -26.05 -33.07 -80.76 -33.31]
5 folds 의 개별 RMSE scores: [3.53 5.1 5.75 8.99 5.77]
5 folds 의 평균 RMSE: 5.829
```

- 5개 폴드 세트에 대해서 교차 검증 수행 ; 평균 RMSE 는 5.829,
`cross_val_score(scoring = "neg_mean_squared_error")`로 반환된 값을 확인해보면 모두 음

05. 다항 회귀와 과(대)적합/과소적합 이해

[다항 회귀 이해]

: 회귀의 독립변수의 단항식이 아닌 2차, 3차 방정식과 같은 다항식으로 표현되는 것을 다항 회귀라고 함. (but, 여전히 선형 회귀!)

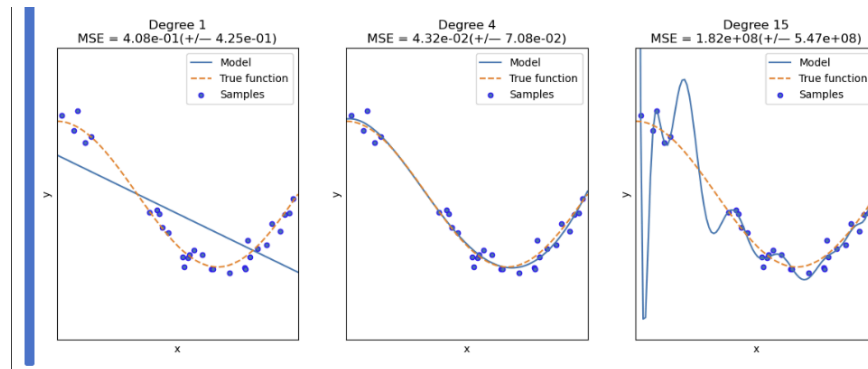
- 사이킷런 : 다항회귀를 위한 클래스 명시적 제공 X, 비선형 함수를 선형 모델에 적용시키는 방법을 사용해 구현
- Polynomial 클래스를 통해 피처를 `polynomial`(다항식) 피처로 변환함
- 사이킷런은 `PolynomialFeatures`로 피처를 변환한 후에 `LinearRegression` 클래스로 다항 회귀를 구현

- 사이킷런의 Pipeline 객체를 이용해 한번에 다항회귀 구현

[다항 회귀를 이용한 과소적합 및 과적합 이해]

다항 회귀의 차수(degree)를 높일수록 학습 데이터에만 너무 맞춘 학습이 이뤄짐 → 정작 테스트 데이터 환경에서는 오히려 예측

정확도가 떨어짐. 즉, 차수가 높아질수록 과적합의 문제가 크게 발생



→ 실선: 다항회귀 예측 곡선, 점선: 실제 데이터 세트 X&Y의 코사인 곡선

⇒ 좋은 예측 모델은 Degree 1과 같이 학습 데이터의 패턴을 지나치게 단순화한 과소적합 모델 X, Degree 15와 같이 모든 학습 데이터의 패턴을 하나하나 감안한 지나치게 복잡한 과적합 모델 X

*학습 데이터의 패턴을 잘 반영하면서도 복잡하지 않은 균형 잡힌(Balanced) 모델을 의미

[편향-분산 트레이드 오프(Bias-Variance Trade off)]

- 고편향성 : 매우 단순화된 모델, 지나치게 한 방향으로 치우친 경향 (Degree1)
- 고분산 : 학습 데이터 하나하나의 특성을 반영하며 매우 복잡한 모델이 되었고 지나치게 높은 변동성을 가지게 됨 (Degree15)
- 편향과 분산은 한쪽이 높으면 한쪽이 낮아지는 경향이 있음
- 높은 편향/낮은 분산 : 과소적합 ↔ 낮은 편향/높은 분산 : 과적합

06. 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

- 비용함수는 학습 데이터의 잔차 오류 값을 최소로 하는 RSS 최소화 방법과 과적합을 방지하기 위해 회귀 계수 값이 커지지 않도록 하는 방법이 서로 균형을 이뤄야 함.

$$\text{비용 함수 목표} = \text{Min}(\text{RSS}(W) + \alpha * \|W\|_2^2)$$

- α : 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터
→ α 값을 크게 하면 과적합 개선, α 값을 작게 하면 학습 데이터 적합을 개선
- 규제 (Regularization) : 비용 함수에 α 값으로 페널티를 부여해 회귀계수 값의 크기를 감소시켜 과적합을 개선하는 방법 (L2, L1)
- L2 규제 : W의 제곱에 대해 페널티 부여 → 릿지(Ridge) 회귀
- L1 규제 : W의 절댓값에 대해 페널티 부여 → 라소 회귀

[릿지 회귀]

주요 생성 파라미터 : α

- α 값의 변화에 따른 피처의 회귀 계수 값

[47]:

	$\alpha:0$	$\alpha:0.1$	$\alpha:1$	$\alpha:10$	$\alpha:100$
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

- α 값이 증가하면서 회귀 계수가 지속적으로 작아짐을 알 수 있지만, 릿지 회귀의 경우에는 회귀 계수를 0으로 만들지는 않음

[라소 회귀]

: W의 절댓값에 페널티를 부여하는 L1 규제를 선형 회귀에 적용한 것

- L2 규제가 회귀 계수의 크기를 감소시키는 데 반해, L1 규제는 불필요한 회귀 계수를 급격하게 감소시켜 0으로 만들고 제거

[63]:

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.789725	3.703202	2.498212	0.949811	0.000000
CHAS	1.434343	0.955190	0.000000	0.000000	0.000000
RAD	0.270936	0.274707	0.277451	0.264206	0.061864
ZN	0.049059	0.049211	0.049544	0.049165	0.037231
B	0.010248	0.010249	0.009469	0.008247	0.006510
NOX	-0.000000	-0.000000	-0.000000	-0.000000	0.000000
AGE	-0.011706	-0.010037	0.003604	0.020910	0.042495
TAX	-0.014290	-0.014570	-0.015442	-0.015212	-0.008602
INDUS	-0.042120	-0.036619	-0.005253	-0.000000	-0.000000
CRIM	-0.098193	-0.097894	-0.083289	-0.063437	-0.000000
LSTAT	-0.560431	-0.568769	-0.656290	-0.761115	-0.807679
PTRATIO	-0.765107	-0.770654	-0.758752	-0.722966	-0.265072
DIS	-1.176583	-1.160538	-0.936605	-0.668790	-0.000000

[엘라스틱넷 회귀]

: L2 규제와 L1 규제를 결합한 회귀

→ 엘라스틱넷은 라쏘 회귀가 서로 상관관계가 높은 피쳐들의 경우에 이들 중에서 중요 피쳐만을 선택하고 다른 피쳐들은 모두 회귀 계수를 0으로 만드는 성향이 강함

- ElasticNet 클래스 : 주요 파라미터 (alpha, l1_ratio)

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.574162	3.414154	1.918419	0.938789	0.000000
CHAS	1.330724	0.979706	0.000000	0.000000	0.000000
RAD	0.278880	0.283443	0.300761	0.289299	0.146846
ZN	0.050107	0.050617	0.052878	0.052136	0.038268
B	0.010122	0.010067	0.009114	0.008320	0.007020
AGE	-0.010116	-0.008276	0.007760	0.020348	0.043446
TAX	-0.014522	-0.014814	-0.016046	-0.016218	-0.011417
INDUS	-0.044855	-0.042719	-0.023252	-0.000000	-0.000000
CRIM	-0.099468	-0.099213	-0.089070	-0.073577	-0.019058
NOX	-0.175072	-0.000000	-0.000000	-0.000000	-0.000000
LSTAT	-0.574822	-0.587702	-0.693861	-0.760457	-0.800368
PTRATIO	-0.779498	-0.784725	-0.790969	-0.738672	-0.423065
DIS	-1.189438	-1.173647	-0.975902	-0.725174	-0.031208

→ alpha 0.5일 때 RMSE가 5.467로 가장 좋은 예측 성능

** 규제 선형 회귀의 가장 대표적인 기법 (릿지, 라쏘, 엘라스틱넷 회귀) 중 어떤 것이 가장 좋은지는 상황에 따라 다름.

[선형 회귀 모델을 위한 데이터 변환]

선형 회귀 모델 : 피쳐값, 타깃값의 분포가 정규분포인 형태 선호

- 사이킷런을 이용해 피쳐 데이터 세트에 적용하는 변환 작업
1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환 or MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행
 2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법 → 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용
 3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포 (log transformation, 로그 변환)

피쳐 데이터 변환 방법 → (None, None) : 아무런 변환을 하지 않은 원본 데이터,
 ('Standard', None): 표준 정규 분포, ('Standard', 2): 표준 정규 분포를 다시 2차 다항식
 변환, ('MinMax', None) : 최솟값/최댓값 정규화, ('MinMax', 2): 최솟값/최댓값 정규화를
 다시 2차 다항식 변환, ('Log', None): 로그 변환

```
## 변환 유형:None, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:5.788
alpha 1일 때 5 폴드 세트의 평균 RMSE:5.653
alpha 10일 때 5 폴드 세트의 평균 RMSE:5.518
alpha 100일 때 5 폴드 세트의 평균 RMSE:5.330

## 변환 유형:Standard, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:5.826
alpha 1일 때 5 폴드 세트의 평균 RMSE:5.803
alpha 10일 때 5 폴드 세트의 평균 RMSE:5.637
alpha 100일 때 5 폴드 세트의 평균 RMSE:5.421

## 변환 유형:Standard, Polynomial Degree:2
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:8.827
alpha 1일 때 5 폴드 세트의 평균 RMSE:6.871
alpha 10일 때 5 폴드 세트의 평균 RMSE:5.485
alpha 100일 때 5 폴드 세트의 평균 RMSE:4.634

## 변환 유형:MinMax, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:5.764
alpha 1일 때 5 폴드 세트의 평균 RMSE:5.465
alpha 10일 때 5 폴드 세트의 평균 RMSE:5.754
alpha 100일 때 5 폴드 세트의 평균 RMSE:7.635

## 변환 유형:MinMax, Polynomial Degree:2
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:5.298
alpha 1일 때 5 폴드 세트의 평균 RMSE:4.323
alpha 10일 때 5 폴드 세트의 평균 RMSE:5.185
alpha 100일 때 5 폴드 세트의 평균 RMSE:6.538

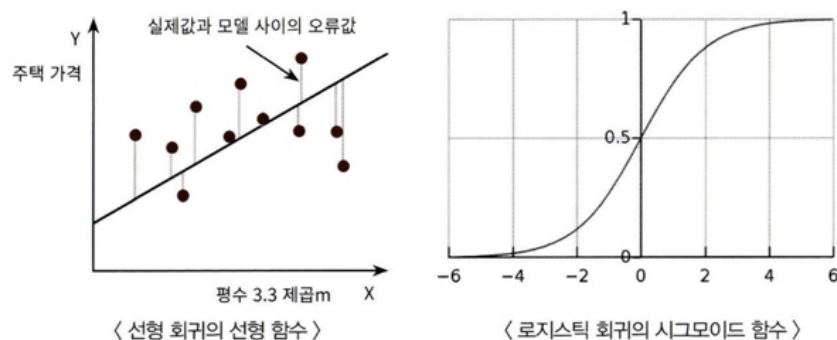
## 변환 유형:Log, Polynomial Degree:None
alpha 0.1일 때 5 폴드 세트의 평균 RMSE:4.770
alpha 1일 때 5 폴드 세트의 평균 RMSE:4.676
alpha 10일 때 5 폴드 세트의 평균 RMSE:4.836
alpha 100일 때 5 폴드 세트의 평균 RMSE:6.241
```

- 결과 : 표준 정규 분포와 최솟값/최댓값 정규화로 피쳐 데이터 세트를 변경해도 성능상의 개선 X
- 다항식 변환은 앞에서 언급한 단점으로 인해 피쳐의 개수가 많을 경우 적용하기 힘들며, 또한 데이터 건수가 많아지면 계산에 많은 시간이 소모되어 적용에 한계가 있음
- 로그 변환 : alpha가 0.1, 1, 10인 경우에 모두 좋은 성능 향상

⇒ 일반적으로 선형 회귀를 적용하려는 데이터 세트에 데이터 값의 분포가 심하게 왜곡되어 있을 경우, 이처럼 로그 변환을 적용하는 것이 좋음.

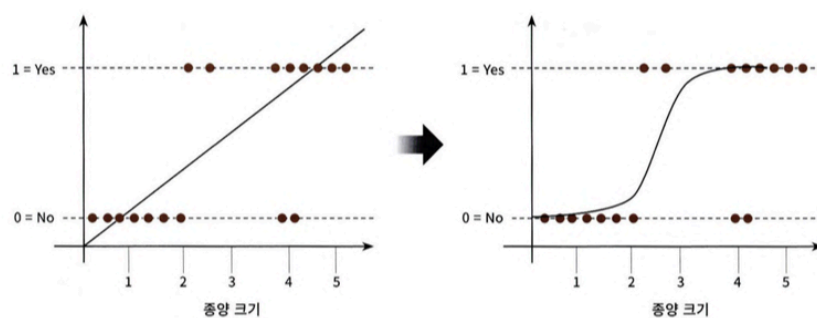
07. 로지스틱 회귀

로지스틱 회귀 : 선형 회귀 방식을 분류에 적용한 알고리즘, 선형 회귀 계열, 학습을 통해 선형 함수 회귀 최적선을 찾는 것이 아니라 시그모이드 (sigmoid) 함수 최적선을 찾고 이 시그모이드 함수의 반환값을 확률로 간주해 확률에 따라 분류를 결정



시그모이드 함수 : $y = 1/(1+e^{-x}) \rightarrow y$ 값은 항상 0과 1 사이 값 반환

로지스틱 회귀는 선형회귀 방식을 기반으로 하되 시그모이드 함수를 이용해 분류를 수행하는 회귀



Logistic Regression 클래스에서 solver 파라미터들 → 최적화

lbfgs	solver의 기본 설정값. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행
liblinear	다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적화(Local Minimum)에 이슈가 있고. 병렬로 최적화할 수 X

newton-cg	좀 더 정교한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려짐
sag	Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용→ 대용량의 데이터에서 빠르게 최적화
saga	sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 함

→ 다양한 solver 값들 중 성능 차이는 적음

****위스콘신 유방암 데이터 세트 → 로지스틱 회귀로 암 여부 판단**

- 선형 회귀 계열의 로지스틱 회귀는 데이터의 정규 분포도에 따라 예측 성능 영향을 받을 수 있음 : 데이터에 먼저 정규 분포 형태의 표준 스케일링을 적용한 뒤에 `train_test_split()`을 이용해 데이터 세트를 분리
- `max_iter` : solver로 지정된 최적화 알고리즘이 최적 수렴할 수 있는 최대 반복 횟수

```
solvers = ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga']
```

여러 개의 solver 값별로 LogisticRegression 학습 후 성능 평가

for solver in solvers:

```
    lr_clf = LogisticRegression(solver=solver, max_iter=600)
```

```
    lr_clf.fit(X_train, y_train)
```

```
    lr_preds = lr_clf.predict(X_test)
```

```
    lr_preds_proba = lr_clf.predict_proba(X_test)[:,-1]
```

accuracy와 roc_auc 측정

```
    print('solver:{0}, accuracy:{1:.3f}, roc_auc:{2:.3f}'.format(solver,
                                                                accuracy_score(y_test, lr_preds),
                                                                roc_auc_score(y_test, lr_preds_proba)))
```

```
solver:lbfgs, accuracy:0.977, roc_auc:0.995
solver:liblinear, accuracy:0.977, roc_auc:0.995
solver:newton-cg, accuracy:0.977, roc_auc:0.995
solver:sag, accuracy:0.977, roc_auc:0.995
solver:saga, accuracy:0.977, roc_auc:0.995
```

liblinear와sag, saga일 경우에 정확도가 0.982, ROC—AUC가 0.995로 lbfgs나 newton-cg보다 상대적인 성능 수치가 약간 높지만, 데이터 세트가 워낙 작기 때문에 크게

의미 있지 X

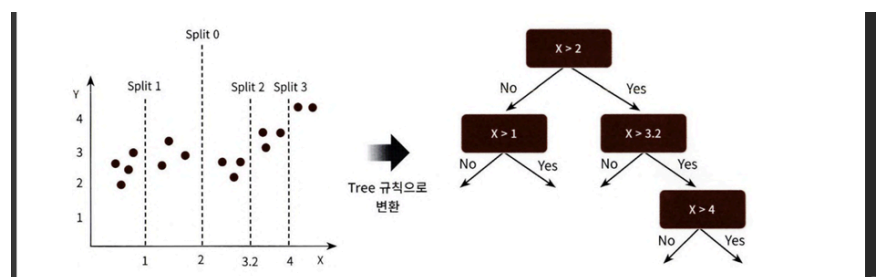
- Solver와 maxjter 외에 사이킷런 LogisticRegression 클래스의 주요 하이퍼 파라미터 : penalty, C
- penalty : 규제(Regularization)의 유형을 설정 → 'l2'로 설정 시 L2 규제, 'l1'으로 설정 시 L1 규제
- C : 규제 강도를 조절하는 alpha 값의 역수 → C값이 작을수록 규제 강도가 큼

로지스틱 회귀는 가볍고 빠르지만, 이진 분류 예측 성능도 뛰어남 → 로지스틱 회귀를 이진 분류의 기본 모델로 사용하는 경우가 많고, 희소한 데이터 세트 분류에도 뛰어난 성능을 보여서 텍스트 분류에서도 자주 사용됨

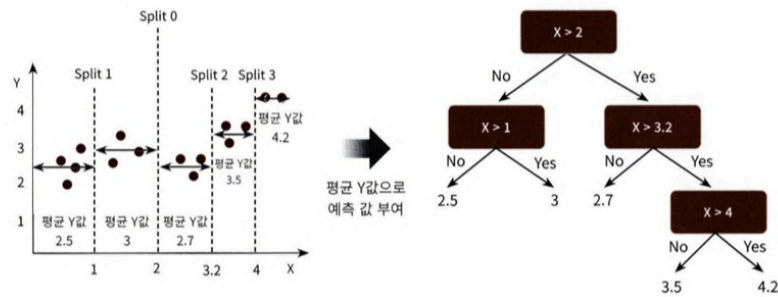
08. 회귀 트리

- 트리 기반의 회귀 : 회귀 트리를 이용하는 것 → 회귀를 위한 트리를 생성하고 이를 기반으로 회귀 예측을 하는 것
- 분류 트리 : 특정 클래스 레이블을 결정 ↔ 회귀 트리: 리프 노드에 속한 데이터 값의 평균값을 구해 회귀 예측값을 계산

[회귀 트리의 동작 방식]



- 데이터 세트의 X 피처를 결정 트리 기반으로 분할 → 지니 계수에 따라 분할.(왼쪽 그림)
- 루트 노드를 split 0 기준으로 분할 → split1,split2 규칙 노드로 분할 → split2는 다시 split3 규칙 노드로 트리 규칙으로 변환 (오른쪽 그림)



- 리프 노드 생성 기준에 부합하는 트리 분할 완료 : 리프 노드에 소속된 데이터 값의 평균 값을 구해서 최종적으로 리프 노드에 결정값으로 할당함.

트리 알고리즘은 CART(Classification And Regression Tree)에 기반 → 분류, 회귀 모두 가능

DecisionTree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

****RandomForestRegressor**을 이용해 보스턴 주택 가격 예

- 회귀 트리 Regressor 클래스는 선형 회귀와 다른 처리 방식이므로 회귀 계수를 제공하는 coef_ 속성 X
- feature_importances → 모델의 피처별 중요도 시각화

다양한 유형의 회귀 트리 생성, 보스턴 주택 가격 예측

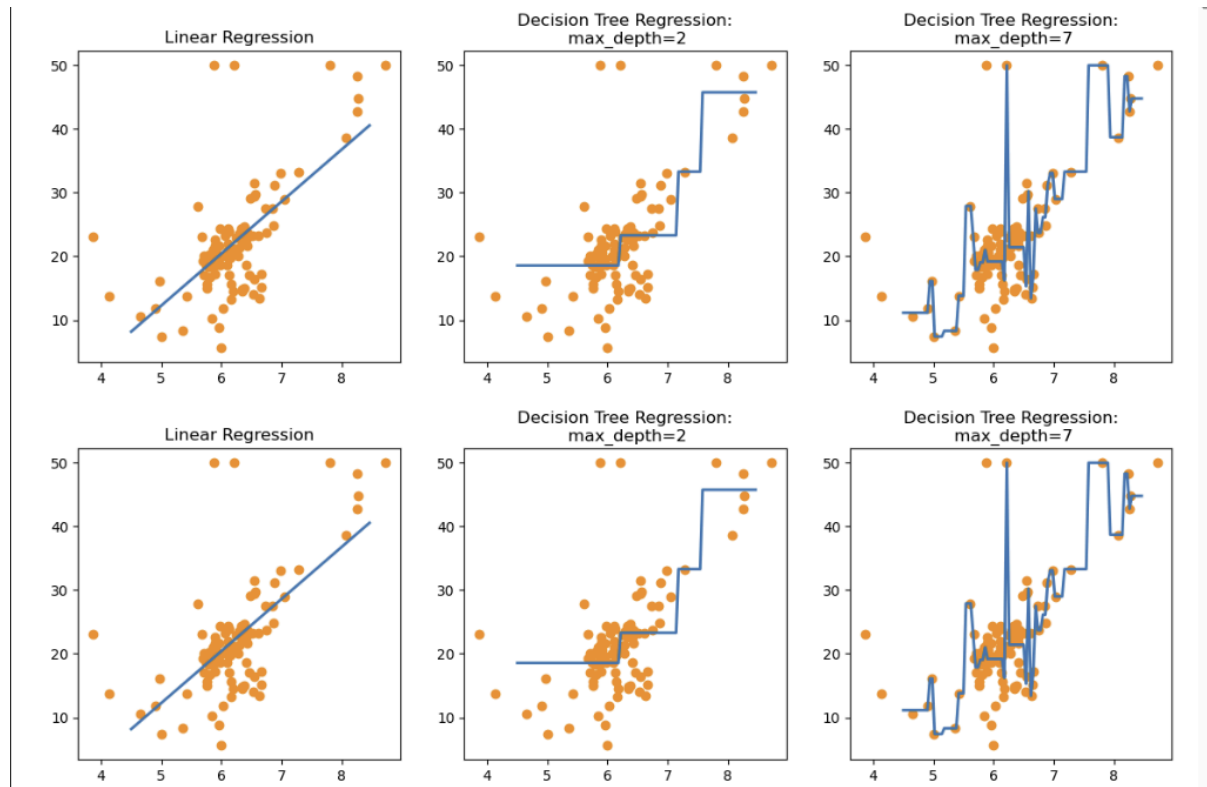
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

dt_reg = DecisionTreeRegressor(random_state=0, max_depth=4)
rf_reg = RandomForestRegressor(random_state=0, n_estimators=1000)
gb_reg = GradientBoostingRegressor(random_state=0, n_estimators=1000)
xgb_reg = XGBRegressor(n_estimators=1000)
lgb_reg = LGBMRegressor(n_estimators=1000)

# 트리 기반의 회귀 모델을 반복하면서 평가 수행
```

```
models = [dt_reg, rf_reg, gb_reg, xgb_reg, lgb_reg]
for model in models:
    get_model_cv_prediction(model, X_data, y_target)
```

LinearRegression, DecisionTreeRegressor의 max_depth를 2,7로 해서 학습된 Regressor에서 예측한 Price 회귀선



→ 선형 회귀 : 직선으로 예측 회귀선 표현

↔ 회귀 트리 : 분할되는 데이터 지점에 따라 브랜치를 만들면서 계단 형태로 회귀선을 만듦.

- DecisionTreeRegressor의 max_depth = 7 : 학습 데이터 세트의 이상치 데이터도 학습하며 복잡한 계단 형태의 회귀선을 만들어 과적합이 되기 쉬운 모델이 됨